

Thesis Proposal: A Decentralized and Adaptive Framework for Distributed LLM Inference on Heterogeneous Edge Devices

[ΥΠΟΤΙΤΛΟΣ ΕΓΓΡΑΦΟΥ]

ΔΕΛΗΜΠΑΛΤΑΔΑΚΗΣ ΓΡΗΓΟΡΙΟΣ
[ΕΠΩΝΥΜΙΑ ΕΤΑΙΡΕΙΑΣ] | [ΔΙΕΥΘΥΝΣΗ ΕΤΑΙΡΕΙΑΣ]

Contents

Main Idea.....	2
Motivation	2
Inspiration.....	2
Petals [5].....	2
Adaptive and Resilient Model-Distributed Inference [4]	3
Model Distributed Inference for LLMs (MDI-LLM) [1]	3
State of the Art and Research Gaps	3
Centrally Coordinated Systems	3
Client Driven and Larger Scale Decentralized Systems.....	3
System Architecture	4
Model Partitioning.....	4
Client Role.....	4
Server Chain	4
Distributed Hash Table (DHT).....	4
Adaptive Layer Allocation Strategy	4
Phase 1: Initial Chain Formation	5
Phase 2: Adaptive Re-allocation (AR-MDI Inspired)	5
Phase 3: Opportunistic Takeover and Self-Healing	5
Implementation Strategy	5
Tools and Frameworks	5
Development Methodology	6
Evaluation.....	6
References	7

Main Idea

Η δημιουργία ενός πλήρως αποκεντρωμένου framework για την εκτέλεση συνεργατικού LLM model-distributed inference σε ένα peer-to-peer δίκτυο από ετερογενή edge devices. Το framework κάνει partition to LLM στο επίπεδο των Transformer Layers και οι “server” nodes του δικτύου αποφασίζουν δυναμικά ποια layers θα φιλοξενήσουν. Αντίθετα με υφιστάμενες προσεγγίσεις, που βασίζονται σε έναν κεντρικό coordinator για την κατανομή του μοντέλου, προτείνεται ένα self-organizing δίκτυο από server nodes. Με αυτόν τον τρόπο, οι ίδιοι οι servers προσαρμόζουν δυναμικά τα LLM layers που φιλοξενούν, δημιουργώντας μια αλυσίδα υπολογισμού, που προσαρμόζεται αυτόματα σε αλλαγές των διαθέσιμων πόρων και μετρήσεις απόδοσης.

Motivation

Το deployment μοντέρνων LLMs σε edge devices αποτελεί μια σημαντική δοκιμασία λόγω του μεγέθους τους, καθιστώντας το αδύνατον, στις περισσότερες περιπτώσεις, να τρέξουν σε resource constrained συσκευές, χωρίς την εξάρτηση από κάποιον remote cloud server.

Παρόλα αυτά, η εκτέλεση AI/ML μοντέλων σε edge devices είναι επιθυμητή για λόγους ιδιωτικότητας, συνεχής διαθεσιμότητας - ακόμα και χωρίς πρόσβαση στο διαδίκτυο - και για την μετρίαση του ανερχόμενου κόστους των cloud resources. Συμπληρωματικά, είναι κρίσιμο ένα σύστημα συνεργατικής εκτέλεσης ενός LLM να είναι αποδοτικό, αξιοποιώντας πλήρως τους διαθέσιμους πόρους, να έχει υψηλό ρυθμό παραγωγής tokens, προσφέροντας ένα ικανοποιητικό user experience και να είναι αξιόπιστο και ανθεκτικό σε node failures.

Για την επίτευξη αυτού του στόχου έχουν προταθεί τεχνικές παραλληλοποίησης για την διαμοίραση του υπολογιστικού φόρτου εκτέλεσης LLMs σε ένα δίκτυο από διασυνδεδεμένες συσκευές. Τεχνικές που βασίζονται στην διαμοίραση στο επίπεδο διακριτών Transformer Layers (model parallelism) [1], στο επίπεδο των Attention Heads [2] και στο επίπεδο των FFN layers (tensor parallelism) [3]. Παρόλο που αυτές οι τεχνικές αποτελούν ένα βήμα προς την σωστή κατεύθυνση, τα παραπάνω distributed συστήματα βασίζονται σε έναν κεντρικό coordinator node [4] [2] ή cloud server [3], που εισάγει ένα single point of failure, υπονομεύοντας τον στόχο της αξιοπιστίας και συνεχής διαθεσιμότητας.

Inspiration

Το προτεινόμενο framework βασίζεται σε state-of-the-art concepts του distributed deep learning.

Petals [5]

Τα θεμέλια της πρότασης της διπλωματικής βασίζονται στο Petals project, ένα αποκεντρωμένο συνεργατικό σύστημα για inference και fine-tuning LLMs με μεγάλο αριθμό παραμέτρων (100B+). Υιοθετείται η peer-to-peer φιλοσοφία του, οι server load balancing τεχνικές του, η χρήση Distributed Hash Table (DHT) για network wide information sharing καθώς επίσης και τα optimizations που εφαρμόζει, όπως dynamic blockwise quantization πριν την μετάδοση των activations, και χρήση compressed model weights.

Adaptive and Resilient Model-Distributed Inference [4]

Ο βασικός μηχανισμός για το αποκεντρωμένο server-driven layer allocation της διπλωματικής, βασίζεται στον lightweight, δυναμικό model allocation αλγόριθμο που παρουσιάζεται στο AR-MDI. Ο αλγόριθμος του paper αυτού, είναι κατάλληλος για την προτεινόμενη προσέγγιση, καθώς δεν βασίζεται σε κάποιον κεντρικό coordinating node, αντίθετα οι worker nodes προσαρμόζουν τον αριθμό των layers που φιλοξενούν βάση των διαθέσιμων πόρων τους και των καταγεγραμμένων computing και communication delays τους. Ωστόσο, το AR-MDI έχει σχεδιαστεί για την εκτέλεση DNNs, οπότε η υιοθέτηση του θα πρέπει να προσαρμοστεί κατάλληλα λαμβάνοντας υπόψιν τις ιδιαιτερότητες των LLMs και της autoregressive φύση τους.

Model Distributed Inference for LLMs (MDI-LLM) [1]

Το άρθρο αυτό εστιάζει στο LLM model distributed inference σε edge devices, τονίζοντας την καταλληλότητα του model parallelism (ή αλλιώς pipeline parallelism), σε αντίθεση με data/tensor parallelism. Παρουσιάζονται δοκιμασίες και λύσεις για την autoregressive φύση των LLMs καθώς επίσης και την ανάγκη για KV Caching σε κάθε node. Συγκεκριμένα προτείνουν έναν μηχανισμό recurrent pipeline parallelism για την αποφυγή idle nodes και rotating KV Caches για την εναλλαγή μεταξύ των Key/Value matrices των διαφορετικών samples, που εκτελούνται στο pipeline, σε κάθε node. Ωστόσο, το MDI-LLM χρησιμοποιεί έναν κεντρικό “starter” node ως coordinator του δικτύου, ο οποίος αρχικοποιεί τους “secondary” worker nodes.

State of the Art and Research Gaps

Centrally Coordinated Systems

Μια κυρίαρχη προσέγγιση στην υλοποίηση συστημάτων distributed LLM inference για edge device networks, είναι η χρήση μιας κεντροποιημένης οντότητας για την κατάτμηση του μοντέλου και το coordination των nodes. Τα συστήματα αυτά είναι προσαρμοστικά με την έννοια ότι καθορίζουν μια αποδοτική στρατηγική deployment για μια δεδομένη κατάσταση του δικτύου και των διαθέσιμων πόρων, με τρόπο όμως κεντροποιημένο.

Συγκεκριμένα, το EdgeShard [6] χρησιμοποιεί έναν κεντροποιημένο scheduler που λύνει ένα model partitioning πρόβλημα, με δυναμικό προγραμματισμό. Βασίζεται σε offline profiling του δικτύου και των nodes και υπολογίζει ένα βέλτιστο fixed deployment plan για να ελαχιστοποιήσει το latency. Η προσέγγιση αυτή έχει δύο βασικά μειονεκτήματα: Ο κεντρικός scheduler συνιστά ένα single point of failure και το σύστημα δεν μπορεί να προσαρμοστεί δυναμικά σε αλλαγές του δικτύου χωρίς την επανεκτέλεση του χρονοβόρου αλγορίθμου βελτιστοποίησης.

Ομοίως, λύσεις όπως το AdaptLink [3] και MDI-LLM [1] βασίζονται σε έναν κεντρικό coordinator για την συλλογή πληροφοριών και μετρήσεων από συμμετέχοντες worker nodes και τον προσδιορισμό του βέλτιστου layer assignment.

Client Driven and Larger Scale Decentralized Systems

Στο Petals project [5], που αποτελεί κυρίαρχη έμπνευση της διπλωματικής, οι servers είναι παθητικοί, δημοσιεύοντας απλά τα layers που φιλοξενούν. Ο client είναι υπεύθυνος για την σύνθεση του βέλτιστου μονοπατιού inference μέσα στο δίκτυο. Παρόλο που πρόκειται για ένα πραγματικά αποκεντρωμένο σύστημα, επιβαρύνει υπολογιστικά τον client με περίπλοκους pathfinding υπολογισμούς. Μια κατάλληλη προσαρμογή και εφαρμογή της

αποκεντρωμένης λογικής κατανομής του μοντέλου του AR-MDI [4], θα μπορούσε να είναι πιο αποδοτική.

Συμπληρωματικά το Petals έχει σχεδιαστεί για το deployment large scale LLMs (100B+ parameters), έχοντας υψηλές απαιτήσεις για collaborating servers (16GB CPU RAM, 8GB GPU RAM, 100 Mbit/s bandwidth) και clients (12GB RAM, 25Mbit/s bandwidth). Ένα πραγματικά αποκεντρωμένο peer-to-peer σύστημα συνεργατικού LLM inference για edge devices, δεν εντοπίζεται στην βιβλιογραφία.

System Architecture

Το προτεινόμενο σύστημα λειτουργεί ως ένα peer-to-peer δίκτυο από resource constrained edge devices, που εκτελούν συνεργατικά το inference ενός LLM. Η νοημοσύνη του συστήματος χωρίζεται μεταξύ των συμμετεχόντων nodes, τα οποία δημιουργούν αυτόματα μια υπολογιστική αλυσίδα και προσαρμόζονται δυναμικά σε μεταβολές του δικτύου.

Model Partitioning

Το LLM χωρίζεται στο επίπεδο των Transformer layers, για λόγους απλότητας και επειδή ανάλογα το μοντέλο, ένα ολόκληρο Transformer layer δεν καταναλώνει πολλούς πόρους (πχ. για το Llama 3.1 8B ένα Transformer layer καταλαμβάνει ~350MB μνήμης).

Client Role

Ένας client node αναλαμβάνει την εκτέλεση του embedding και των τελικών output layers τοπικά. Αυτή η επιλογή, απλοποιεί τις απαιτήσεις των server nodes και μειώνει τα απαιτούμενα network hops.

Συμπληρωματικά, ο client εντοπίζει την «κεφαλή» του server chain και κάνει ερωτήματα στο DHT του συστήματος για να επιβεβαιώσει, πως όλα τα layers είναι διαθέσιμα προς εκτέλεση, και υπάρχει ένα διαθέσιμο inference path.

Server Chain

Οι server nodes του δικτύου δημιουργούν μια λογική ακολουθιακή αλυσίδα επεξεργασίας των Transformer layers που αναλαμβάνουν, με κάθε server να έχει επίγνωση του successor του. Αυτή η δομή παρέχει με ξεκάθαρη ακολουθία για την διαδικασία του layer allocation και την ροή των δεδομένων κατά το inference.

Distributed Hash Table (DHT)

Λαμβάνοντας έμπνευση από το Petals [5], χρησιμοποιείται ένα DHT, που λειτουργεί ως το “bulletin board” του δικτύου. Οι servers δημοσιεύουν σε αυτό βασικές πληροφορίες, όπως το network id τους, τον successor τους στην αλυσίδα, το εύρος των Transformer layers που φιλοξενούν και βασικές μετρικές (διαθέσιμη μνήμη, computational latency, communication delay με τον επόμενο node).

Adaptive Layer Allocation Strategy

Το δίκτυο των servers εφαρμόζει ένα adaptive layer allocation strategy εμπνευσμένο από το AR-MDI [4], επιτρέποντας του να οργανώνεται αυτόματα και να εξισορροπεί τον υπολογιστικό φόρτο δυναμικά. Η διαδικασία προσαρμογής, πραγματοποιείται μεταξύ ολοκληρωμένων inference responses, διαφυλάσσοντας την σταθερότητα του inference path κατά την παραγωγή κειμένου.

Phase 1: Initial Chain Formation

Ο πρώτος server γίνεται η κεφαλή της αλυσίδας. Όταν ένας νέος server εισάγεται στο δίκτυο, εντοπίζει μέσω του DHT την «ουρά» της αλυσίδας και τοποθετείται αμέσως μετά από αυτήν, ενημερώνοντας την προηγούμενη ουρά.

Ξεκινώντας από την κεφαλή, κάθε server αναλαμβάνει όσα layers του επιτρέπει η διαθέσιμη μνήμη του, αφήνοντας περιθώριο για την αυξανόμενη KV Cache.

Τέλος, κάθε server ανακοινώνει στο DHT τα layer που έχει αναλάβει.

Phase 2: Adaptive Re-allocation (AR-MDI Inspired)

Αφού ολοκληρωθεί το πρώτο inference task, με το greedy allocation που περιεγράφηκε παραπάνω, το δίκτυο έχει συλλέξει performance data και μπορεί να πραγματοποιήσει ένα re-allocation των layers για να βελτιστοποιήσει το throughput.

Συγκεκριμένα, κάθε server έχει μετρήσει την απόδοση του (computational delay και communication delay με τον successor του) και το processing rate του (number of parameters processed per second). Συμπληρωματικά, η ουρά της αλυσίδας έχει υπολογίσει το συνολικό processing rate του συστήματος αθροίζοντας τα partial rates των προηγούμενων servers. Πριν το επόμενο inference task, οι servers εκτελούν έναν lightweight αποκεντρωμένο layer re-allocation αλγόριθμο αξιοποιώντας αυτές τις πληροφορίες.

Ακολουθώντας την λογική του AR-MDI [4], κάθε server υπολογίζει το νέο βέλτιστο μερίδιο παραμέτρων του μοντέλου που μπορεί να αναλάβει, βάσει της συνεισφοράς του στο συνολικό processing rate του δικτύου. Έπειτα, τα Transformer layers ανατίθενται εκ νέου με την σειρά στην αλυσίδα, προσεγγίζοντας αυτό το βέλτιστο μερίδιο παραμέτρων που υπολογίστηκε.

Phase 3: Opportunistic Takeover and Self-Healing

Σκοπός του δικτύου είναι να βελτιώνει την απόδοση του συνεχώς και να διαθέτει δυνατότητες ανάκαμψης, σε περιπτώσεις αποτυχίας κόμβων.

Εάν ένας νέος, πιο ισχυρός κόμβος εισαχθεί στο δίκτυο μπορεί να εκτιμήσει την τρέχουσα απόδοση των ενεργών servers και να αντικαταστήσει κάποιον υπάρχοντα κόμβο αν υπολογίσει πως μπορεί να αυξήσει το συνολικό processing rate ή να αποσυμφορήσει κάποιον κόμβο που βρίσκεται στο όρια της διαθέσιμης μνήμης του.

Συμπληρωματικά, αν κάποιος server αποτύχει και σπάσει η υπολογιστική αλυσίδα, οι υπόλοιποι servers μπορούν να συντονιστούν μέσω του DHT και να αναδιανείμουν ορφανά layers μεταξύ τους. Εναλλακτικά, κάποιο backup server θα μπορούσε να ενεργοποιηθεί για να αναλάβει τα layers του αποτυχημένου node.

Implementation Strategy

Tools and Frameworks

Το συνολικό framework θα υλοποιηθεί σε Python, χρησιμοποιώντας την βιβλιοθήκη PyTorch. Το κύρια μοντέλο γύρω από το οποίο θα χτιστεί το framework θα μπορούσαν να είναι τα Llama 3.2 1B και 3B, καθώς έχουν μοντέρνα αρχιτεκτονική και ένα διαχειρίσιμο μέγεθος, κατάλληλο για ταχεία πειράματα.

Συγκεκριμένα, όμοια με το MDI-LLM [1], μπορούν να χρησιμοποιηθούν υλοποιήσεις των μοντέλων μέσω της βιβλιοθήκης [lit-gpt](#).

Το κρίσιμο peer-to-peer δίκτυο και οι αποκεντρωμένες λειτουργίες μπορούν να υλοποιηθούν χρησιμοποιώντας την βιβλιοθήκη [hivemind](#), όμοια με το Petals project [5].

Development Methodology

Η μεθοδολογία ανάπτυξης μπορεί να ακολουθήσει τρεις φάσεις.

Αρχικά, θα υλοποιηθεί η server-side λογική, δηλαδή η δημιουργία της λογικής αλυσίδας υπολογισμού των Transformer layers και η υλοποίηση του προσαρμοστικού layer allocation αλγορίθμου.

Στην συνέχεια, θα υλοποιηθεί η εφαρμογή των clients, με την απαραίτητη λογική φόρτωσης και εκτέλεσης των αρχικών και τελικών layers του μοντέλου, ανίχνευσης της αλυσίδας των servers στο τοπικό δίκτυο και πραγματοποίησης ερωτημάτων στο DHT για την επικύρωση της πλήρης κάλυψης του μοντέλου.

Τέλος, θα ενσωματωθούν μηχανισμοί fault-tolerance. Από την πλευρά των servers, θα υλοποιηθούν μηχανισμοί για τον αυτόματο εντοπισμό αποτυχιών, επιτρέποντας στο δίκτυο να ανακάμψει δυναμικά. Η ανάκαμψη θα επιτυγχάνεται είτε με την ανακατανομή των "ορφανών" layers στους υπόλοιπους ενεργούς κόμβους είτε με την ενεργοποίηση ενός backup αντικαταστάτη server. Για να διασφαλιστεί η απρόσκοπτη συνέχιση της διαδικασίας παραγωγής κειμένου, θα αναπτυχθεί και η αντίστοιχη λογική από την πλευρά του client για την ανακατασκευή της KV Cache στον νέο αντικαταστάτη server.

Evaluation

Για τον έλεγχο της λειτουργικότητας και απόδοσης του προτεινόμενου framework, μπορούν να πραγματοποιηθούν πειράματα σε ένα σύνολο από virtual machines ή Raspberry Pis/Jetsons/Laptops (εάν αυτά είναι διαθέσιμα), εξομοιώνοντας ένα ρεαλιστικό ετερογενές edge computing environment.

Τα baselines για την σύγκριση του framework θα μπορούσαν να είναι: ένα στατικό greedy partitioning των layers, φορτώνοντας το μοντέλο στους αρχικά διαθέσιμους servers μέχρι να γεμίσει η μνήμη τους, μια naïve κατανομή ίσου αριθμού layers σε κάθε server και η εκτέλεση ενός μικρότερου LLM σε ένα μόνο device.

Οι μετρικές απόδοσης του framework μπορούν να είναι το throughput (tokens/second) και κατανομή της χρήσης μνήμης μεταξύ των nodes του δικτύου.

References

- [1] D. M. a. H. S. a. E. Koyuncu, «Model-Distributed Inference for Large Language Models at the Edge,» *arXiv*, 13 May 2025.
- [2] D. K. a. R. K. a. I. Koutsopoulos, «Large Language Model Partitioning for Low-Latency Inference at the Edge,» *arXiv*, 2025.
- [3] Z. C. K. G. M. Z. a. H. H. Y. X. Hu, «AdaptLink: A Heterogeneity-Aware Adaptive Framework for Distributed MLLM Inference,» *Openreview.net*, 2024.
- [4] E. K. a. H. S. P. Li, «Adaptive and Resilient Model-Distributed Inference in Edge Computing Systems,» *IEEE Open Journal of the Communications Society*, 2023.
- [5] A. B. e. al., «Petals: Collaborative Inference and Fine-tuning of Large Models,» 2022.
- [6] { . Z. a. J. C. a. X. S. a. Z. Cui, «EdgeShard: Efficient LLM Inference via Collaborative Edge Computing,» *arXiv*, 2024.